Развертывание AI сервера

в ОС Astra Linux 1.7

для сохранения форматирования текста при работе с документом (копирование/вставка фрагментов) используйте Chromium и Yandex браузеры

ollamA-webui AI

Учебный материал предоставил:

https://github.com/UAR-2020/ **www.uar@gmail.com**

2026

Содержание:

I. Создание необходимой программной среды

На тестовом сервере 1 :

1. установка Ollama сервера (наличие Nvidia не обязательно)……..……3
2. создание пула необходимых моделей для Ollama сервера……..… 8
3. создание переносимого архива моделей для Ollama сервера……..….... 9
4. скачивание и архивирование докера с Ollama сервером……...….. 10
5. скачивание и архивирование докера с WebUI сервером……..10
6. скачивание NVIDIA Container Toolkit…….... 10

II. Развертывание созданной программной среды

На тестовом сервере 2 :

1. установка docker.io и установка драйвера и NVIDIA Container Toolkit..... 11
2. распаковка и загрузка(load) докера с Ollama сервером…….….. 12
3. распаковка переносимого архива моделей для докера с Ollama сервером…....12
4. распаковка и загрузка(load) докера с WebUI сервером……..…….13
5. запуск(run) докеров с Ollama сервером и WebUI сервером…… 13

III. Подключение к созданному AI серверу через браузер

На любой тестовой рабочей станции :

1. подключение к AI серверу…….…….17
2. принцип работы и настройка моделей…….. 17
3. устранение ошибки 500 на клиенте WebUI…….…. 17

IV. Создание и подключение cli ассистента ии(ai) к AI серверу

На любой тестовой рабочей станции :

1. создание и добавление скрипта в ~/.bashrc……...…….…….. 18
2. вызов клиента и основы работы с ним…...…….…. 19

V. Прочее

1. Отключение рассуждений для моделей через WebUI и CLI…….…..……21
2. Удаление докеров и их данных…….. 22
3. Ограничение потребления оперативной памяти (RAM) докерами…….. 23

ollamA-webui AI

На тестовом сервере 1 :

Рекомендации по установке Ollama взяты с ресурса URL

https://docs.ollama.com/linux

Установка Ollama (автоматически скачает и настроит службу)

Для установки Ollama выполните следующую команду:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Ручная установка Ollama

[*** **все версии Ollama на расположены по адресу**

https://github.com/ollama/ollama/releases ,

учитывайте это при формировании адреса при скачивании пакета (!!!)

]

Если вы обновляете программу с предыдущей версии, сначала следует удалить старые библиотеки `sudo rm -rf /usr/lib/ollama`.

Скачайте и распакуйте пакет:

```
curl -fsSL https://ollama.com/download/ollama-linux-amd64.tar.zst \
    | sudo tar x -C /usr
```

Запуск олламы:

```
ollama serve
```

В другом терминале убедитесь, что Ollama запущена:

```
ollama -v
```

Добавление Ollama в качестве сервиса (рекомендуется)

Создайте пользователя и группу для Ollama:

ollamA-webui AI

```
sudo useradd -r -s /bin/false -U -m -d /usr/share/ollama ollama
sudo usermod -a -G ollama $(whoami)
```

Создайте файл службы в `/etc/systemd/system/ollama.service`:

```
[Unit]
Description=Ollama Service
After=network-online.target

[Service]
ExecStart=/usr/bin/ollama serve
User=ollama
Group=ollama
Restart=always
RestartSec=3
Environment="PATH=$PATH"

[Install]
WantedBy=multi-user.target
```

Затем запустите сервис:

```
sudo systemctl daemon-reload
sudo systemctl enable ollama
```

Установка драйверов CUDA (необязательно).

Скачайте и установите CUDA.Убедитесь, что драйверы установлены, выполнив следующую команду, которая должна вывести подробную информацию о вашей видеокарте:

```
nvidia-smi
```

ollamA-webul AI

При установке из apt выбрать необходимую версию, к примеру:

```
sudo apt install nvidia-driver-575
```

В нашем случае отсутствует необходимость в установке проприетарных драйверов вместо Nouveau от Astra Linux в связи с тем, что будет выполнятся только скачивание необходимых моделей для Ollama для создания необходимой программной среды

Запуск Ollama

Запустите Ollama и убедитесь, что она работает:

```
sudo systemctl start ollama
sudo systemctl status ollama
```

Настройка

Для настройки установки Ollama можно отредактировать файл службы systemd или переменные среды, выполнив команду:

```
sudo systemctl edit ollama
```

В качестве альтернативы, создайте файл переопределения вручную в `/etc/systemd/system/ollama.service.d/override.conf`:

```
[Service]
Environment="OLLAMA_DEBUG=1"
```

Обновление

ollamA-webul AI

Обновите Ollama, повторно запустив скрипт установки:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Или повторно загрузив Ollama:

```
curl -fsSL https://ollama.com/download/ollama-linux-
amd64.tar.zst \
    | sudo tar x -C /usr
```

Установка определенных версий

Используйте `OLLAMA_VERSION`переменную среды в скрипте установки, чтобы установить определенную версию Ollama, включая предварительные версии. Номера версий можно найти на [странице релизов](#) .Например:

```
curl -fsSL https://ollama.com/install.sh | OLLAMA_VERSION=0.30.0
sh
```

Просмотр журналов

Чтобы просмотреть журналы работы Ollama в качестве службы, выполните следующую команду:

```
journalctl -e -u ollama
```

Удаление Ollama

Удалите сервис Ollama:

```
sudo systemctl stop ollama
```

ollamA-webui AI

```
sudo systemctl disable ollama
sudo rm /etc/systemd/system/ollama.service
```

Удалите библиотеки Ollama из каталога lib (либо `/usr/local/lib`, `/usr/lib`, либо `/lib`):

```
sudo rm -r $(which ollama | tr 'bin' 'lib')
```

Удалите исполняемый файл ollama из каталога bin (либо `/usr/local/bin`, `/usr/bin`, либо `/bin`):

```
sudo rm $(which ollama)
```

Удалите загруженные модели, а также пользователя и группу службы Ollama:

```
sudo userdel ollama
sudo groupdel ollama
sudo rm -r /usr/share/ollama
```

ollamA-webui AI

При выборе видеокарты необходимо помнить, что она не должна быть слабее **CC 5.0** (Compute Capability 5.0+)
Видеокарта должна иметь микроархитектуру не ниже **Maxwell** (например, серия NVIDIA **gtx1060 и выше** **для моделей 4-6 GB** и обладать необходимым объемом VRAM для ВСЕХ задач, в том числе обеспечение прочих программ и процессов выделением VRAM)
При выборе драйвера должна быть заявлена поддержка **CUDA 12 или 13** (драйверы NVIDIA версии **535 и выше**)

Скачайте интересующую версию модели. На странице https://ollama.com/search найдите необходимую модель, выбрав ее из популярных или вбейте поиск, к примеру «qwen3.5»

Нажмите на ссылку qwen3.5

(https://ollama.com/library/qwen3.5)

Прокрутите страницу вниз и выберите View all →

И страница предстанет в таком виде:

...	...	...	...
qwen3.5:4b 2a654d98e6fb 2 months ago	3.4GB	256K	Text, Image
qwen3.5:9b latest 6488c96fa5fa 2 months ago	6.6GB	256K	Text, Image
...	...	...	...
qwen3.5:2b-bf16 64148169aba8 1 month ago	4.6GB	256K	Text, Image
...	...	...	...
qwen3.5:122b-a10b-q4_K_M 8b9d11d807c5 · 2 months ago	81GB	256K	Text, Image

OllamA-webui AI

Обычные игровые карты обладают 6-8 Gb VRAM.

Загружаясь в VRAM, модель забирает не только свой размер, но и память под контекст, который в зависимости от настроек может забирать дополнительно 0,5-2 GB VRAM в зависимости от ее настроек (как регулировать будет изложено дополнительно).

Если все это выходит за пределы доступного VRAM, то задействует уже RAM и CPU ПК и начинает тормозить при формировании ответа и забирать ресурсы программ и процессов ПК. Иногда при этом она и вовсе делится ½ GPU и ½ CPU.

Поэтому выберем модель, которая вписывается в перечисленные параметры.

Нажимаем на ссылку qwen3.5:2b-bf16 (4.6GB; 256K; Text, Image) и увидим:

qwen3.5:2b-bf16

11.9M Downloads Updated 1 month ago

Qwen 3.5 is a family of open-source multimodal models that delivers exceptional utility and performance.

CLI cURL Python JavaScript

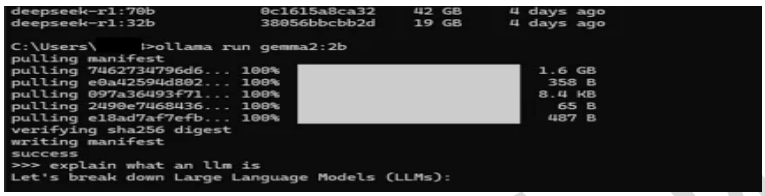
ollama pull qwen3.5:2b-bf16

(pull -сохранить, run - запустить)

Выполним в CLI:

ollama pull qwen3.5:2b-bf16

будет выглядеть так (ps : это от другой модели 1,6 GB, не такой интеллектуальной)



https://losst.pro/kak-polzovatsya-ollama-dlya-nachinayushhih

- по этой ссылке можно наглядно посмотреть все этапы скачивания прочего в графическом представлении

Выкачаем подобной командой все необходимые модели.

Посмотреть какие модели в наличие локально можно с помощью команды `ollama list`

Это выдаст на экран список наших моделей

Теперь, **создадим архив наших моделей для переноса на другой сервер**:

(выполните команду в терминале, чтобы упаковать всю папку в один сжатый архив)

tar -czvf **ollama_models_all.tar.gz** -C /usr/share/ollama/.ollama models

(название архива задается Вами самостоятельно и содержит все установленные модели)

ollamA-webui AI

Скачиваем докеры Ollama и WebUI

docker pull ollama/ollama

docker pull ghcr.io/open-webui/open-webui:main

(чтобы удалить модель, к примеру не хотите ее в архив добавлять, выполните:

docker rm имя_модели) Прочие команды — обратитесь к документации Ollama

Сохраним эти образы в архивы:

docker save -o ollama_image.tar ollama/ollama

docker save -o webui_image.tar ghcr.io/open-webui/open-webui:main

NVIDIA Container Toolkit — это специальный набор инструментов, который позволяет контейнерам (например, Docker или Podman) использовать физическую видеокарту (GPU) вашего компьютера для ускорения вычислений. [1]

NVIDIA Container Toolkit можно скачать с URL:

https://github.com/NVIDIA/nvidia-container-toolkit/releases

Состоит из:

NOTE: This release is a unified release of the NVIDIA Container Toolkit that consists of the following packages:

- libnvidia-container 1.19.1
- nvidia-container-toolkit 1.19.1

Скачиваем доступную на момент написания инструкции крайнюю версию ПО https://github.com/NVIDIA/nvidia-container-toolkit/releases/download/v1.19.1/nvidia-container-toolkit_1.19.1_deb_amd64.tar.gz

ollamA-webui AI

На тестовом сервере 2 :

Установим пакет Docker и добавим юзера в нужную группу:

```
sudo apt install docker.io
sudo usermod -aG docker $USER
```

Про требования к видеокарте и драйверу (стр.7 данного документа) и про установку драйвера (стр. 4 даного документа) было изложено ранее.

Если драйвер из репозитория по каким то причинам не подходит, то произвести установку драйвера с сайта производителя следуя по образу и подобию инструкции по установке СПО AxxonNext(Intellect-X), используя уже версию драйвера предложенную вашему видеоадаптеру.

Начиная с модельного ряда NVIDIA RTX 5xxx необходимо устанавливать версию драйвера с упоминанием слова «open» (nvidia-driver-5xx-open)

Выполним установку NVIDIA Container Toolkit

Распакуем скаченный файл:

```
tar -xzvf nvidia-container-toolkit_1.19.0_deb_amd64.tar.gz
```

Перейдем по пути:

```
cd release-v1.19.0-stable/packages/ubuntu18.04/amd64
```

Выполним установку пакетов:

```
sudo dpkg -i *.deb
```

Необходимо произвести интеграцию инструментов в docker.io

Пропишем runtime NVIDIA в системные настройки Docker и перезапустим службу:

```
sudo nvidia-ctk runtime configure --runtime=docker
```

OllamA-webui AI

```
sudo systemctl restart docker
```

Выполним команду импорта образа Ollama :

```
sudo docker load -i ollama_image.tar
538812a4b9bd: Loading layer [==================================================>]  80.65MB/80.65MB
f236f023d347: Loading layer [==================================================>]  337.7MB/337.7MB
afceac86a52f: Loading layer [==================================================>]  44.95MB/44.95MB
d2870ea00208: Loading layer [==================================================>]  6.091GB/6.091GB
Loaded image: ollama/ollama:latest
```

Создайте на сервере папку для моделей (если её еще нет) и распакуйте туда вашу «россыпь» файлов из переносимого архива моделей:

sudo mkdir -p /var/lib/docker/volumes/ollama_data/_data/models

sudo tar -xzvf ollama_models_all.tar.gz -C /var/lib/docker/volumes/ollama_data/_data/

(это рекомендуется делать с выключенным докером с Ollama сервером, - мы его импортировали, но еще не запускали))

```
models/
models/blobs/
models/blobs/sha256-832dd9e00a68dd83b3c3fb9f5588dad7dcf337a0db50f7d9483f310cd292e92e
models/blobs/sha256-8893e08fa9f91f7dc39e24d27bdfaece4e9c86bb3269293ff8cea6cba98c872d
models/blobs/sha256-9371364b27a52acac9d87f88bd93c9db1174d8d6ec57f6888925cdc1788871ff
...
models/blobs/sha256-60e05f2100071479f596b964f89f510f057ce397ea22f2833a0cfe029bfc2463
models/blobs/sha256-1e65450c30670713aa47fe23e8b9662bdf4065e81cc8e3cbfaa98924fcc0d320
models/manifests/
models/manifests/registry.ollama.ai/
models/manifests/registry.ollama.ai/library/
models/manifests/registry.ollama.ai/library/llama3.1/
models/manifests/registry.ollama.ai/library/llama3.1/8b
models/manifests/registry.ollama.ai/library/qwen2.5-coder/
models/manifests/registry.ollama.ai/library/qwen2.5-coder/7b
models/manifests/registry.ollama.ai/library/gemma2/
models/manifests/registry.ollama.ai/library/gemma2/9b
models/manifests/registry.ollama.ai/library/qwen2.5/
models/manifests/registry.ollama.ai/library/qwen2.5/7b
models/manifests/registry.ollama.ai/library/deepseek-coder/
models/manifests/registry.ollama.ai/library/deepseek-coder/6.7b-instruct-q5_K_M
models/manifests/registry.ollama.ai/library/qwen3.5/
models/manifests/registry.ollama.ai/library/qwen3.5/4b-q8_0
```

Зададим права: sudo chown -R 755:755 /var/lib/docker/volumes/ollama_data/_data/models

(или) sudo chown -R 997:973 /var/lib/docker/volumes/ollama_data/_data/models

ollamA-webui AI

Выполним команду импорта образа WebUI :

```
sudo docker load -i open_webui_image.tar
```

Создадим изолированную виртуальную сеть `docker`:

```
sudo docker network create ai-cluster-network
```

По умолчанию это сеть типа bridge (мост) и необходима для:

Безопасности и изоляции): Контейнеры внутри сети **`ai-cluster-network`** полностью изолированы от других контейнеров в системе, которые в эту сеть не входят.

Организации кластера: используется для объединения инфраструктуры искусственного интеллекта (например, сервера Ollama, веб-интерфейса Open WebUI и агентов) в один логический контур.

Предоставления выделенного имени (DNS): Контейнеры в одной пользовательской сети могут общаться друг с другом по своим именам (например, ***`http://ollama_server:11434`***), а не по IP-адресам.

Запустим Ollama сервер:

```
sudo docker run -d \
  --name ollama_server \
  --network ai-cluster-network \
  --gpus all \
  --ipc=host \
  --pids-limit 150 \
  -e OLLAMA_HOST=0.0.0.0 \
  -e OLLAMA_NUM_PARALLEL=1 \
  -e OLLAMA_MAX_LOADED_MODELS=1 \
  -e OLLAMA_KEEP_ALIVE=0 \
  -e OLLAMA_SCHED_SPREAD=false \
  -e NVIDIA_VISIBLE_DEVICES=all \
  -e NVIDIA_DRIVER_CAPABILITIES=compute,utility \
  -v ollama_data:/root/.ollama \
  -p 11434:11434 \
  --restart always \
  ollama/ollama:latest
```

OllamA-webui AI

Посмотрим как запустился докер Ollama и пробросилось ли в него GPU:

```
sudo docker logs ollama_server | grep -i -E "gpu|cuda|detect|nvidia"
```

```
time=2026-05-22T09:51:58.557Z level=INFO source=routes.go:1802 msg="server config"
env="map[CUDA_VISIBLE_DEVICES: GGML_VK_VISIBLE_DEVICES: GPU_DEVICE_ORDINAL: HIP_VISIBLE_DEVICES:
HSA_OVE
RRIDE_GFX_VERSION: HTTPS_PROXY: HTTP_PROXY: NO_PROXY: OLLAMA_CONTEXT_LENGTH:0 OLLAMA_DEBUG:INFO
OLLAMA_DEBUG_LOG_REQUESTS:false OLLAMA_EDITOR: OLLAMA_FLASH_ATTENTION:false OLLAMA_GPU_OVER
HEAD:0 OLLAMA_HOST:http://0.0.0.0:11434 OLLAMA_KEEP_ALIVE:0s OLLAMA_KV_CACHE_TYPE:
OLLAMA_LLM_LIBRARY: OLLAMA_LOAD_TIMEOUT:5m0s OLLAMA_MAX_LOADED_MODELS:1 OLLAMA_MAX_QUEUE:512
OLLAMA_MAX_
TRANSFER_STREAMS:4 OLLAMA_MODELS:/root/.ollama/models OLLAMA_MULTIUSER_CACHE:false
OLLAMA_NEW_ENGINE:false OLLAMA_NOHISTORY:false OLLAMA_NOPRUNE:false OLLAMA_NO_CLOUD:false
OLLAMA_NUM_PAR
ALLEL:1 OLLAMA_ORIGINS:[http://localhost https://localhost http://localhost:* https://localhost:*
http://127.0.0.1 https://127.0.0.1 http://127.0.0.1:* https://127.0.0.1:* http://0.0.0.0
https://0.0.0.0 http://0.0.0.0:* https://0.0.0.0:* app://* file://* tauri://* vscode-webview://*
vscode-file://*] OLLAMA_REMOTES:[ollama.com] OLLAMA_SCHED_SPREAD:false OLLAMA_VULKAN:false
ROCR_VISIBLE_DEVICES: http_proxy: https_proxy: no_proxy:]"
time=2026-05-22T09:51:58.557Z level=INFO source=routes.go:1804 msg="Ollama cloud disabled: false"
time=2026-05-22T09:51:58.557Z level=INFO source=images.go:517 msg="total blobs: 0"
time=2026-05-22T09:51:58.557Z level=INFO source=images.go:524 msg="total unused blobs removed: 0"
time=2026-05-22T09:51:58.558Z level=INFO source=routes.go:1864 msg="Listening on [::]:11434
(version 0.24.0)"
time=2026-05-22T09:51:58.559Z level=INFO source=runner.go:67 msg="discovering available GPUs..."
time=2026-05-22T09:51:58.594Z level=INFO source=server.go:433 msg="starting runner"
cmd="/usr/bin/ollama runner --ollama-engine --port 39275"
time=2026-05-22T09:52:00.095Z level=INFO source=model_recommendations.go:177 msg="model
recommendations cache sleep scheduled" wait=3h51m34.847979847s consecutive_failures=0
time=2026-05-22T09:52:01.542Z level=INFO source=server.go:433 msg="starting runner"
cmd="/usr/bin/ollama runner --ollama-engine --port 44433"
time=2026-05-22T09:52:04.176Z level=INFO source=runner.go:106 msg="experimental Vulkan support
disabled.  To enable, set OLLAMA_VULKAN=1"
time=2026-05-22T09:52:04.176Z level=INFO source=server.go:433 msg="starting runner"
cmd="/usr/bin/ollama runner --ollama-engine --port 38377"
time=2026-05-22T09:52:04.300Z level=INFO source=types.go:42 msg="inference compute"
id=GPU-63b55c3a-bf8f-e304-ee7d-19ea877c06d1 filter_id="" library=CUDA compute=6.1
name=CUDA0 description="NVIDIA P106-100" libdirs=ollama,cuda_v12 driver=12.9
pci_id=0000:01:00.0 type=discrete total="6.0 GiB" available="5.9 GiB"
time=2026-05-22T09:52:04.300Z level=INFO source=routes.go:1914 msg="vram-based default context"
total_vram="6.0 GiB" default_num_ctx=4096
```

**Здесь мы наблюдаем информацию о нашем видеоадаптере и дополнительные данные связанные с ним.*

Просмотрим наличие моделей в докере

ollamA-webUI AI

```
sudo docker exec -it ollama_server ollama list
```

NAME	ID	SIZE	MODIFIED
qwen3.5:4b-q8_0	8722f47c2791	5.3 GB	30 hours ago
gemma2:9b	ff02c3702f32	5.4 GB	31 hours ago
llama3.1:8b	46e0c10c039e	4.9 GB	3 days ago
deepseek-coder:6.7b-instruct-q5_K_M	d6a1221fb5ff	4.8 GB	4 days ago
qwen2.5:7b	845dbda0ea48	4.7 GB	8 days ago
qwen2.5-coder:7b	dae161e27b0e	4.7 GB	8 days ago

Запустим WebUI сервер:

```
sudo docker run -d \
 --name open_webui \
 --network ai-cluster-network \
 -p 3050:8080 \
 -e WEBUI_NAME="Интерфейс предоставил UAR-2020" \
 -e OLLAMA_BASE_URL="http://ollama_server:11434" \
 -e OPENAI_API_BASE_URL="" \
 -e ENABLE_OLLAMA_API=True \
 -e OFFLINE_MODE=true \
 -e HF_HUB_OFFLINE=1 \
 -e WEBUI_AUTH=false \
 -e WEBUI_LOG_LEVEL=WARNING \
 --log-opt max-size=10m \
 --log-opt max-file=3 \
 -v open_webui_data:/app/backend/data \
 --restart always \
 ghcr.io/open-webui/open-webui:main
```

OllamA-webui AI

```
administrator@docker-test:~$ nvidia-smi
Fri May 22 14:50:41 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 575.64.03              Driver Version: 575.64.03      CUDA Version: 12.9     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA P106-100                Off |   00000000:01:00.0 Off |                  N/A |
| 78%   37C    P0             27W /  120W |    5443MiB /   6144MiB |      2%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A            1771      G   /usr/lib/xorg/Xorg                        4MiB |
|    0   N/A  N/A            5024      C   /usr/bin/ollama                        5436MiB |
+-----------------------------------------------------------------------------------------+
administrator@docker-test:~$
```

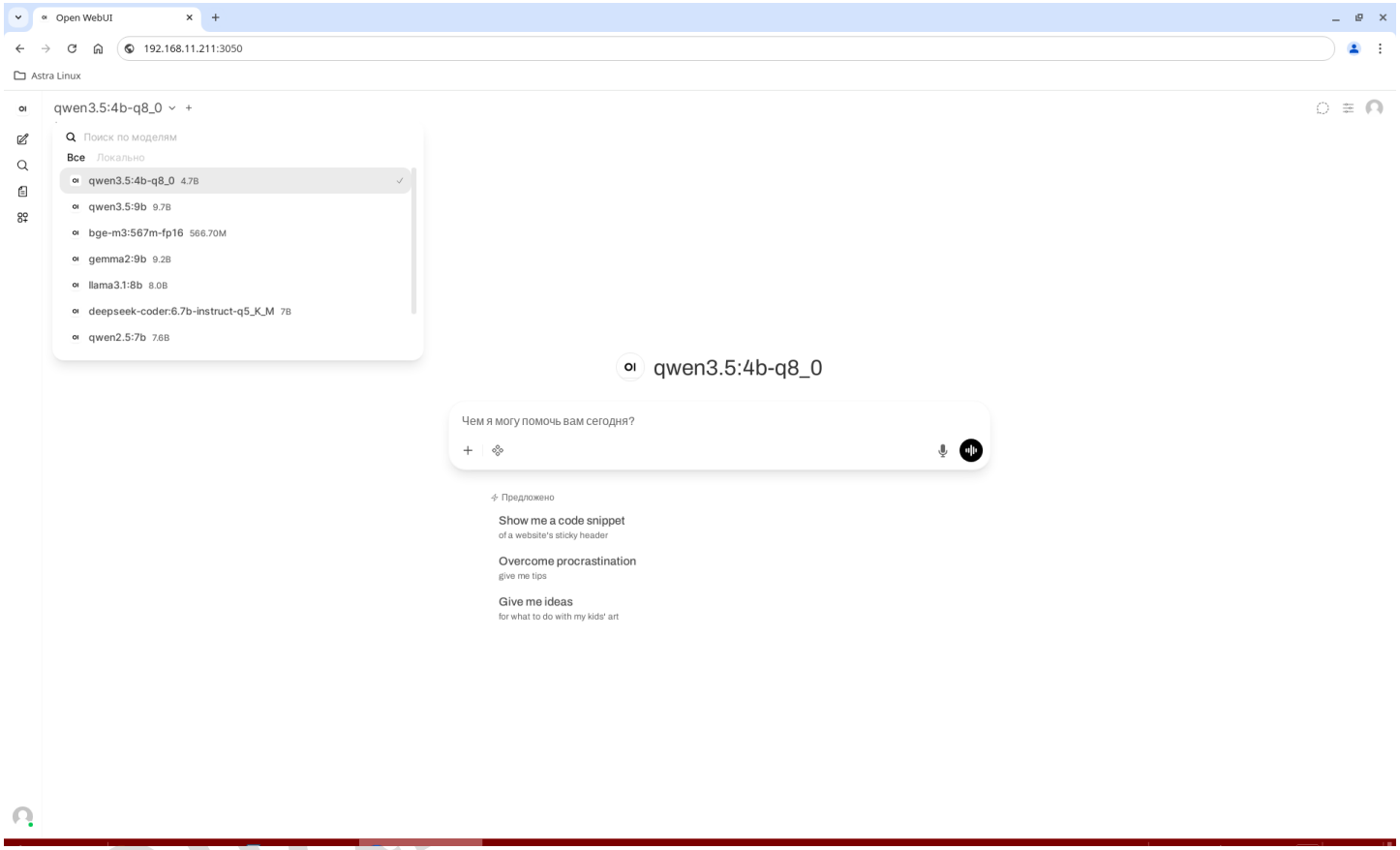
В конфигурационном файле Ollama у нас выставлены настройки загружать за один раз только одну модель для того, чтобы вторая не загрузилась и не полезла в CPU и тормозить I/O ОС (это стандартная ситуация при нехватке GPU, т. е. для нормальной работы мы не можем одновременно загрузить две модели превышающие 8 GB GPU (на некоторых видеоадаптерах установлено 6 GB (рисунок сверху) и модели нужно выбирать еще тщательнее) либо нужно ставить две видеокарты и более мощный БП. Также предусмотрена мгновенная выгрузка модели, чтобы оппонент смог без задержек подключиться к своей модели.

На изображении выше (вывод nvidia-smi) показана загрузка модели qwen2.5:7b 4.7 GB, которая с обработкой контекста и генерацией в среднем занимает 5.5 GB(!!!). Здесь не приведен пример qwen3.5:9b 6,6 GB — она при полном развертывании занимает 8,11 GB (!!!) из 8 Gb GPU Nvidia RTX5060 (нужно быть внимательнее, т. е. обращать внимание, чтобы модель не вылезла за пределы GPU в CPU) и хватило GPU на прочие ресурсы. Модель qwen3.5:4b-q8_0 5.3 GB лучше вписывается в предложенное (7,44 GB

ollamA-webUI AI

при полной загрузке), никто не запрещает использовать модель qwen3.5:9b 6.6 GB, просто нужно следить чтобы это не вылезло за пределы GPU (а вылазиет!), ведь, к примеру, вебраузер и прочее ПО может отъедать так нужную нам GPU, необходимо прийти к такой ситуации когда всем хватит GPU и останется немного на возможные прыжки в использовании GPU.

Зайдем в созданный нами механизм через веб-браузер: IP нашего сервера и порт 3050



Выбираем нужную модель и работаем (вставляем текст и опрашиваем).

Некоторые модели qwen3.5 могут обрабатывать изображение и видео, но для этого нужно использовать другую программу-конвейер (решение найдено, в разработке).

Можно работать с нескольких мест одновременно, но запрос оппонента будет сразу выполнен после обработки ранее поступившего запроса. В настройках конфигурационного файла мы отключили вход по паролю, чтобы включить его и дать возможность пользователям самостоятельно задавать пароли — обратитесь к документации WebUI.

При ошибке 500 в браузере — очистите браузер (удалить данные веб-сёрфига и куки).

Для возможности обращаться к модели (предлагается прописать ее имя и

изменять по необходимости) в CLI создадим скрипт:

nano ~/.bashrc и в конец добавить (исправьте адрес IP в скрипте на IP адрес сервера Ollama):

```
ai() {
    echo -e "== Ollama-WebUI AI для CLI AstraLinux [Интерфейс предоставил Юрий Архипов] =="
    echo -e "   Вставьте текст (Ctrl+Shift+V) и напишите вопросы (или наоборот) и нажмите Enter"
    echo -e "   Можете сразу написать вопрос и нажать Enter (допустимо нажатие несколько Enter)"
    echo -e "   Для отправки в ИИ нажмите Enter, а затем Ctrl+D в любой раскладке:"
    echo "-------------------------------------------------------------------------------"

    # привязка Ctrl+ru/en 'в' к сигналу конца ввода (EOF) для bash
    bind '"\C-v": end-of-file' 2>/dev/null
    bind '"\u0432": end-of-file' 2>/dev/null

    # чтение всего текста
    local prompt=$(cat)

    # возврат стандартного поведение клавиш
    bind '"\C-v": paste' 2>/dev/null

    if [ -z "$(echo "$prompt" | tr -d '[:space:]')" ]; then
        echo "Ошибка: запрос не может быть пустым."
        return 1
    fi
    echo "-------------------------------------------------------------------------------"
    echo -ne "\e[1;34mИИ:\e[0m "

    # отправка на сервер IP:11434
    jq -n --arg p "$prompt" '{model: "qwen3.5:4b-q8_0", prompt: $p, stream: true}' | \
    curl -s -N -X POST -H "Content-Type: application/json" http://192.168.11.211:11434/api/generate -d @- | \
    jq -j --unbuffered '.response? // empty'

    echo -e "\n"
}
alias фш=ai
```

ollamA-webul AI

Еще раз ОБРАТИТЕ ВНИМАНИЕ НА СТРОКУ — там должен быть IP Ollama сервера

curl -s -N -X POST -H "Content-Type: application/json" http://192.168.11.211:11434/api/generate…..

Рекомендуется в скрипте место IP вбить имя **ai** и в /etc/hosts добавить строку:

ai IP_адрес_Ollama_сервера (проверка доступности и отклик api не предусмотрены)

Далее выполним команду source ~/.bashrc

Попросим ИИ, к примеру, написать скрипт.

Запустим скрипт командой `ai`

```
administrator@test:~$ ai
== Ollama-WebUI AI для CLI AstraLinux [Интерфейс предоставил UAR-2020] ==
  Вставьте текст (Ctrl+Shift+V) и напишите вопросы (или наоборот) и нажмите Enter
  Можете сразу написать вопрос и нажать Enter (допустимо нажатие несколько Enter)
  Для отправки в ИИ нажмите Enter, а затем Ctrl+D в любой раскладке:
--------------------------------------------------------------------------------
напиши скрипт проверки доступности серверов и ведения лога когда недоступен. Их адреса
192.168.11.1, 192.168.11.2, 193.168.11.111
--------------------------------------------------------------------------------
ИИ: Конечно! Ниже приведен пример скрипта на Bash для проверки доступности серверов с
указанными IP-адресами и записи лога при недоступности.

```bash
#!/bin/bash

# Файл для хранения логов
LOG_FILE="server_log.txt"

# Функция для записи в лог файл
log_message() {
   echo "$(date +'%Y-%m-%d %H:%M:%S') - $1" >> "$LOG_FILE"
}

# Список IP-адресов серверов
SERVERS=("192.168.11.1" "192.168.11.2" "193.168.11.111")

# Цикл для проверки каждого сервера
for SERVER in "${SERVERS[@]}"; do
   # Проверка доступности сервера с помощью ping
```

ollama-webui AI
```

```
    if ! ping -c 1 "$SERVER" > /dev/null; then
        log_message "Сервер $SERVER недоступен"
    else
        log_message "Сервер $SERVER доступен"
    fi
done

log_message "Проверка завершена."
```

Как использовать скрипт:
1. Сохраните код в файл, например `check_servers.sh`.
2. Дайте ему разрешение на выполнение:
   ```bash
   chmod +x check_servers.sh
   ```
3. Запустите скрипт:
   ```bash
   ./check_servers.sh
   ```

Объяснение:
- **log_message**: Функция для записи сообщений с текущей датой и времени в файл `server_log.txt`.
- **SERVERS**: Массив IP-адресов серверов, которые будут проверяться.
- **ping -c 1 "$SERVER" > /dev/null**: Команда `ping` отправляет один пакет данных к указанному адресу. Оператор `>` перенаправляет вывод в `/dev/null`, чтобы скрыть вывод команды `ping`.
- **if ! ping ... then**: Если сервер недоступен (пинг не прошел), то вызывается функция `log_message` с сообщением о том, что сервер недоступен.
- **log_message "Проверка завершена."**: В конце записи в лог файл добавляется сообщение об окончании проверки.

Этот скрипт можно запускать регулярно через cron для постоянного мониторинга доступности серверов.
administrator@test:~$

Для организации работы группы серверов Ollama для распределения запросов и генерации предлагается использовать ngnix с учетом загруженности GPU (с учетом наших настроек в простое память видеоадаптера высвобождается на минимум).

Некоторые модели часто долго рассуждают и сильно греют видеоадаптер.

ollamA-webul AI

Для отключения рассуждений в WebUI для определенной модели (не все рассуждают)**:**

Решение:

1. перейти к моделям:

   (лицо-профиль в правом верхнем углу → Панель администратора → Настройки → Модели)

   или набрать http://ваш_IP:3050/admin/settings/models
2. найти нужную модель и справа от нее нажать логотип с карандашом (редактирование)
3. Найти строку «Расширенные параметры» и справа от нее нажать «Показать»
4. Найти строку «think (Ollama)» и выбрать для нее «Выключено»
5. Внизу найти и нажать кнопку «Сохранить и обновить»

Для отключения рассуждений в вызываемом скрипте ai:

К примеру, qwen3.5:4b-q8_0 рассуждает по умолчанию и много.
В CLI долго не появляется ответ и рассуждения не выводятся.

Решение:

Для скрипта на странице 18 данного документа [вызывает в CLI доступ к ИИ(ai)] в строку формирования JSON-пакета добавить поле think: false

```
jq -n --arg p "$prompt" '{model: "qwen3.5:4b-q8_0", think: false, prompt: $p, stream: true}'
```

Выполнить `source ~/.bashrc`

Теперь Ollama принудительно пропустит этап генерации мыслей для указанной модели и сразу начнет выводить текстовый ответ в терминал.

В скрипте ai для cli по умолчанию (на странице 18 данного документа) прописана qwen3.5:4b-q8_0. *Выбирайте свою модель, опираясь на рекомендации данного руководства (чтобы хватило ресурсов всем потребителям ОС).*

ollamA-webui AI

Иногда, необходимо начать с чистого листа.

Ниже предоставлены команды, которые обеспечат нам это:

Удаление Ollama и его данных

Решение:

Удалить докер Ollama:

sudo docker stop ollama_server && sudo docker rm ollama_server

(либо удалить без остановки докера командой «sudo docker rm -f ollama_server»)

Удалить только модели (выполняется после удаления докера Ollama):

sudo rm -rf /var/lib/docker/volumes/ollama_data/_data/models/*

И далее, если есть необходимость, стереть абсолютно все данные Ollama (включая логи, метаданные и настройки):

sudo docker volume rm ollama_data

Удаление Open-WebUI и его данных:

Решение:

Удалить докер Open-WebUI:

sudo docker stop open_webui && sudo docker rm open_webui

(либо удалить без остановки докера командой «sudo docker rm -f open_webui»)

И далее, если есть необходимость, стереть абсолютно все данные Open-WebUI (включая логи, метаданные и настройки):

sudo docker volume rm open_webui_data

ollamA-webui AI

На Astra Linux 1.7 «жор» памяти связан со спецификой работы дискового кэша операционной системы (`Page Cache`) и тем, как Docker изолирует процессы.

Когда Ollama запускается, она считывает файл модели с диска в оперативную память (**RAM)**, а затем переносит его в **VRAM** видеокарты. По спецификации Linux, прочитанные данные **намертво оседают в RAM как кэш файловой системы**, чтобы при повторном запросе не читать диск. Поскольку Ollama работает внутри контейнера, Docker приписывает этот системный кэш к потреблению памяти именно вашего контейнера, из-за чего вы и видите 10 (или другое значение) ГБ.

Так как `nvidia-smi` подтверждает, что модель полностью сидит в видеокарте, можно **безопасно забрать у контейнера лишнюю оперативную память** прямо при запуске через `docker run`.

Решение: Ограничение RAM через флаг `docker run`

Поскольку мы запускаем контейнер одиночной командой, добавим в неё флаг `--memory` (или `-m`) и ограничим потребление, например, на уровне **3-4 ГБ**. Этого хватит для работы API-сервера Ollama, но не даст системе раздувать кэш.

На всякий случай предлагается ограничить потребление RAM на уровне размера модели, но необходимость этого только предстоит изучить.

Модифицируйте вашу команду запуска следующим образом:

bash

```
sudo docker run -d \
  --name ollama_server \
  --network ai-cluster-network \
  --gpus all \
  -m 4g \
  --memory-swap 4g \
    -…
```

Описание флагов:

- `-m 4g` — жестко запрещает контейнеру и его процессам использовать более 4 ГБ обычной оперативной памяти.
- `--memory-swap 4g` — отключает возможность сброса данных контейнера в swap-раздел на диске (значение должно быть равно значению `-m`). Это критично на Astra Linux, чтобы контейнер не начал тормозить, пытаясь писать кэш на диск.

ПРОВЕРИМ:

1. Перезапустим контейнер с новыми флагами (через удаление докера — см. стр.22)
2. Проверим реальное потребление памяти контейнером командой:

docker stats ollama_server

Теперь показатель `MEM USAGE` зафиксируется в пределах лимита и не будет расти. На скорость генерации (токены в секунду) это никак не повлияет, так как сами вычисления идут исключительно в VRAM. Это можно применить к прочим докерам.

ollamA-webul AI